

REQUIREMENTS ANALYSIS DOCUMENT

Explore Docker Tool

Subject : Software Engineering Laboratory

Document : Object-Oriented Requirements Analysis

Domain : Containerisation and DevOps Tooling

Standard : IEEE / ISO/IEC/IEEE 29148

1. Introduction

1.1 Brief Explanation of Docker

Docker is an open-source containerisation platform that allows developers and system administrators to package, distribute, and execute applications inside lightweight, portable units called containers. A container bundles an application's code together with all of its runtime dependencies — libraries, configuration files, and environment variables — into a single deployable artifact that runs consistently across different host environments.

Unlike traditional virtual machines, Docker containers share the host operating-system kernel and do not require a full OS installation per workload. This makes them significantly lighter, faster to start, and more resource-efficient. Docker achieves this through Linux kernel features such as namespaces (for process isolation) and cgroups (for resource limiting).

The central components of Docker are: (1) the Docker Engine, which acts as the server-side daemon and REST API; (2) Images, which are read-only templates used to create containers; (3) Containers, which are running instances of those images; (4) Volumes, which provide persistent storage; (5) Networks, which govern container-to-container and container-to-host communication; and (6) the Docker CLI, which is the primary human interface for issuing commands.

1.2 Purpose of the Project

The purpose of this Requirements Analysis Document is to formally analyse the Docker tool using Object-Oriented Analysis (OOA) techniques. Specifically, the document applies the Entity-Boundary-Control (ECB) architectural pattern to identify all relevant objects in the Docker domain, model their responsibilities and collaborations through CRC (Class-Responsibility-Collaboration) cards, define their associations and aggregations, catalogue their attributes, model state-dependent behaviour, and establish inheritance hierarchies. The analysis is based strictly on the object catalogue established in the accompanying ECB Identification document.

1.3 Scope of the System

This analysis covers the following functional scope of the Docker tool:

- Container lifecycle management: creation, starting, stopping, pausing, restarting, and removal of containers.
- Image management: pulling images from registries, building images from Dockerfiles, tagging, pushing, and removing images.
- Volume management: creation, mounting, inspection, and removal of persistent storage volumes.
- Network management: creation, attachment, detachment, inspection, and removal of virtual networks.
- Docker Engine operations: daemon lifecycle, REST API handling, resource limit enforcement, and system-level monitoring.

- User interaction: command issuance via the Docker CLI by Developer and System Administrator roles.
- Registry interaction: authentication, image push, and image pull with remote registries.
- Multi-container orchestration: definition and deployment of applications via Docker Compose files.

The scope does not include Kubernetes orchestration, cloud-provider-specific integrations, Docker Enterprise commercial features, or CI/CD pipeline tooling beyond what Docker natively provides.

1.4 What is Being Analysed

This document performs a complete object-oriented analysis of the Docker tool by examining the objects identified in the ECB document, structured into three architectural layers:

- Entity Objects: Persistent domain objects that represent the core data and state of the system — User, DonorProfile (adapted as DockerUser), Container, Image, Volume, Network, Dockerfile, FoodSafetyGuideline (adapted as DockerSafetyGuideline), Feedback, Rating, PickupRequest, Schedule, Volunteer, DeliveryPartner, DistributionRecord, Beneficiary, Donor, PenaltyRecord, Violation, FoodInspector, InspectionSchedule, Assignment, FoodDonation, SafetyFlag, DisposalRecord, NGO, MatchingCriteria, Location, IssueReport, PickupTask, AvailabilitySchedule, DeliveryTask, DeliveryStatus, PickupStatus, Role, Permission, ActivityLog, Report, DonationRecord, DatabaseRecord, BackupLog, AuditLog, Notification, TrackingLog, StatusHistory, InspectionReport, and FoodSafetyRule.
- Boundary Objects: Interface objects that mediate between the User and the system — all UI forms, dashboards, and views listed in the ECB document.
- Control Objects: Coordinator objects that implement business logic and orchestrate interactions between Entities and Boundaries — all Controller classes listed in the ECB document.

2. Modeling Interactions Using CRC Cards

The following CRC cards are derived strictly from the Entity-Boundary-Control object catalogue in the reference ECB document. Colour coding is used to distinguish object types: blue headers = Entity, teal headers = Boundary, purple headers = Control.

2.1 Entity Objects

2.1.1 User

Class	Responsibilities	Collaborators
User(Entity)	Store user credentials (username, passwordHash, role)	AuthenticationController

	Maintain the authentication state of a system actor	RegistrationController
	Provide identity data to AuthenticationController for login validation	DonorProfile (DockerUser)
	Serve as the base entity for all specialised user types (Developer, Admin)	Role
	Record account creation timestamp and last-login information	Permission
	Support password reset operations via AuthenticationController	

2.1.2 DonorProfile (DockerUser)

Class	Responsibilities	Collaborators
DonorProfile(Entity)	Store extended profile attributes for the Docker user (display name, organisation, contact)	User
	Maintain a reference to the parent User entity for identity resolution	RegistrationController
	Track the user's preferred Docker context and registry credentials	RegistrationFormUI
	Provide profile data to the registration and management UI boundaries	
	Record the history of containers, images, and volumes created by this user	
	Support profile update operations triggered by the RegistrationController	

2.1.3 Credentials

Class	Responsibilities	Collaborators
Credentials(Entity)	Store the hashed password and authentication token for a user account	AuthenticationController
	Provide credential data to AuthenticationController during login validation	User
	Support token refresh and expiry management	
	Maintain a record of failed login attempts for security auditing	
	Store registry authentication tokens for Docker Hub and private registries	

	Flag credentials as compromised or requiring reset	
--	--	--

2.1.4 FoodDonation (DockerResource)

Class	Responsibilities	Collaborators
FoodDonation(Entity)	Represent a generic Docker-managed resource (Container, Image, or Volume instance)	DonationController
	Store resource identifier, name, type classification, and current status	DonationValidationController
	Maintain lifecycle state transitions from creation through to deletion	DonationManagementController
	Provide resource metadata to DonationController for management operations	DonationTrackingController
	Record the creation timestamp and the User who initiated the resource	DonationStatus
	Support inspection queries that return full resource configuration details	Location
	Track association with parent Image, attached Network, and mounted Volume	

2.1.5 FoodItem (ImageLayer)

Class	Responsibilities	Collaborators
FoodItem(Entity)	Represent an individual filesystem layer within a Docker Image	FoodDonation
	Store layer digest (SHA256), size, and the Dockerfile instruction that created it	DonationController
	Maintain the ordered position of the layer within the full image stack	
	Provide layer metadata to the DonationController for image inspection operations	
	Support layer caching determination during incremental image builds	
	Record whether the layer is shared with other images (shared-layer deduplication)	

2.1.6 Donor

Class	Responsibilities	Collaborators
-------	------------------	---------------

Donor(Entity)	Represent the Docker user who submits (creates) a container, image, or volume resource	DonationController
	Store donor identifier, name, organisation, and contact reference	PenaltyRecord
	Maintain the list of resources submitted by this donor in the system	Violation
	Provide donor identity to DonationController for resource ownership tracking	
	Support penalisation records when the donor submits non-compliant resources	
	Record acknowledgment of food safety guidelines (Docker usage policies)	

2.1.7 Location

Class	Responsibilities	Collaborators
Location(Entity)	Store the host machine address, data-centre zone, or network endpoint of a resource	FoodDonation
	Provide location data to the matching system for proximity-based resource allocation	MatchingController
	Maintain geographic or logical coordinates for container deployment targets	DonationController
	Support location filtering in the DonationController's availability queries	
	Record IP address ranges and subnet information for Network resources	
	Store registry endpoint URLs for remote image storage locations	

2.1.8 DonationStatus

Class	Responsibilities	Collaborators
DonationStatus(Entity)	Represent the current lifecycle state of a Docker resource (Created/Running/Paused/Stopped/Deleted)	DonationTrackingController
	Store the state label, entry timestamp, and the actor who triggered the transition	FoodDonation
	Validate that only permitted state transitions are applied	DonationStatusDashboardUI
	Notify DonationTrackingController on every state change event	

	Maintain a full audit trail of all previous states for lifecycle reporting	
	Provide status data to DonationStatusDashboardUI for real-time display	

2.1.9 Feedback

Class	Responsibilities	Collaborators
Feedback(Entity)	Store free-text feedback comments submitted by a User after a completed operation	FeedbackController
	Maintain a reference to the Docker resource (container run, image build) being reviewed	User
	Record the submission timestamp and the authoring User identifier	FoodDonation
	Provide feedback content to FeedbackController for persistence and retrieval	Rating
	Support retrieval of all feedback associated with a specific resource for reporting	
	Link to Rating entity for composite review record	

2.1.10 Rating

Class	Responsibilities	Collaborators
Rating(Entity)	Store a numeric rating score (1–5) submitted by a User for a completed Docker operation	RatingController
	Maintain a reference to the Docker resource receiving the rating	FeedbackController
	Record submission timestamp and the User who issued the rating	FoodDonation
	Provide rating data to RatingController for aggregation and average computation	User
	Support retrieval of all ratings for a given resource for performance reporting	
	Enforce valid rating range constraints (1 to 5 inclusive)	

2.1.11 FoodSafetyGuideline (DockerPolicy)

Class	Responsibilities	Collaborators
-------	------------------	---------------

FoodSafetyGuideline(Entity)	Represent a Docker security or usage policy rule (e.g., disallowed base images, resource limits)	GuidelineController
	Store policy rule identifier, description, severity level, and enforcement date	GuidelinesViewUI
	Provide policy data to GuidelineController for display and acknowledgment tracking	FoodSafetyRule
	Maintain version history of policy rules as they are updated by the inspector role	
	Enforce mandatory acknowledgment before a User may submit new resources	
	Support deactivation of outdated policy rules without deletion	

2.1.12 PickupRequest (TaskRequest)

Class	Responsibilities	Collaborators
PickupRequest(Entity)	Represent a pending task request for a volunteer (e.g., container deployment task)	PickupRequestController
	Store request identifier, location, required time slot, and food/resource type	PickupTaskController
	Maintain the assignment status of the request (Unassigned, Assigned, Completed)	PickupTask
	Provide request data to PickupRequestController for listing and filtering	Location
	Support real-time distance and resource detail display in PickupRequestsListUI	
	Track which Volunteer has accepted the request once assigned	

2.1.13 Schedule

Class	Responsibilities	Collaborators
Schedule(Entity)	Store time slot availability data for a Volunteer or Delivery Partner	PickupSchedulerController
	Maintain a list of available days and time ranges	AvailabilityController
	Provide schedule data to PickupSchedulerController for task assignment optimisation	Volunteer
	Support conflict detection when multiple tasks overlap within the same time slot	DeliveryPartner

	Record modification history with timestamps for accountability	
	Enable marking of specific slots as blocked or unavailable	

2.1.14 Volunteer

Class	Responsibilities	Collaborators
Volunteer(Entity)	Store volunteer identifier, name, contact information, and availability schedule	PickupSchedulerController
	Maintain the list of PickupTasks currently assigned to this volunteer	TaskAssignmentController
	Provide volunteer data to TaskAssignmentController for pickup coordination	AvailabilityController
	Track volunteer performance rating and task completion history	PickupTask
	Support availability update operations via AvailabilityController	Schedule
	Record any issues reported during active pickup tasks	

2.1.15 DeliveryPartner

Class	Responsibilities	Collaborators
DeliveryPartner(Entity)	Store delivery partner identifier, name, vehicle information, and contact data	DistributionController
	Maintain the list of DeliveryTasks currently assigned to this partner	DeliveryTaskController
	Provide partner data to DistributionController for delivery coordination	DeliveryTask
	Track partner availability and current delivery status	DeliveryStatus
	Support task acceptance and rejection operations	
	Record delivery confirmations including timestamp and destination verification	

2.1.16 DistributionRecord

Class	Responsibilities	Collaborators
DistributionRecord(Entity)	Record the complete details of a completed resource distribution event	DistributionController

	Store distribution identifier, resource type, quantity, source, and destination	ReportController
	Maintain the timestamp of completion and the responsible DeliveryPartner	FoodDonation
	Provide distribution data to DistributionController for reporting	Beneficiary
	Support linkage to the parent FoodDonation resource for full lifecycle tracing	
	Enable audit queries for administrators via ReportController	

2.1.17 Beneficiary

Class	Responsibilities	Collaborators
Beneficiary(Entity)	Store beneficiary identifier, name, address, and resource requirements	DistributionController
	Represent the recipient endpoint for delivered Docker resources	DistributionRecord
	Provide beneficiary data to DistributionController for matching and routing	
	Maintain delivery receipt records upon successful distribution	
	Support multiple resource type requirements per beneficiary	
	Track historical receipt records for reporting purposes	

2.1.18 PenaltyRecord

Class	Responsibilities	Collaborators
PenaltyRecord(Entity)	Store penalty identifier, type, severity, and date issued	PenaltyController
	Maintain a reference to the Donor who incurred the penalty	ViolationAssessmentController
	Link to the InspectionReport or Violation that triggered the penalty	Donor
	Provide penalty data to PenaltyController for enforcement and notification	Violation
	Support escalation from warning to formal penalty based on severity classification	InspectionReport

	Record the NGO administrator who authorised the penalty action	
--	--	--

2.1.19 Violation

Class	Responsibilities	Collaborators
Violation(Entity)	Store violation identifier, type, description, and detected date	ViolationAssessmentController
	Represent a detected breach of Docker policy or food safety guidelines	PenaltyController
	Maintain a reference to the resource (FoodDonation) where the violation was found	FoodDonation
	Provide violation data to ViolationAssessmentController for severity scoring	InspectionReport
	Support linkage to PenaltyRecord when formal penalisation is triggered	
	Record whether the violation was remedied or remains unresolved	

2.1.20 FoodInspector (DockerInspector)

Class	Responsibilities	Collaborators
FoodInspector(Entity)	Store inspector identifier, name, assigned zone, and availability status	InspectorAssignmentController
	Represent the role responsible for conducting resource inspections	InspectionController
	Maintain the list of InspectionSchedules assigned to this inspector	InspectionSchedule
	Provide inspector data to InspectorAssignmentController for task routing	Assignment
	Track inspection outcomes completed by this inspector	
	Support reassignment when the primary inspector is unavailable	

2.1.21 InspectionSchedule

Class	Responsibilities	Collaborators
InspectionSchedule (Entity)	Store the scheduled date, time, and resource reference for a planned inspection	InspectorAssignmentController

	Maintain assignment status (Scheduled, In Progress, Completed, Cancelled)	InspectionController
	Provide schedule data to InspectorAssignmentController for coordination	Assignment
	Support rescheduling when the assigned inspector becomes unavailable	FoodInspector
	Record completion timestamps and outcome references upon inspection finish	
	Link to the parent Assignment entity for full assignment chain traceability	

2.1.22 Assignment

Class	Responsibilities	Collaborators
Assignment(Entity)	Store assignment identifier, type (pickup or inspection), and status	TaskAssignmentController
	Maintain references to the assigned actor (Volunteer, Inspector) and the target resource	InspectorAssignmentController
	Provide assignment data to TaskAssignmentController and InspectorAssignmentController	PickupTask
	Track creation timestamp and the NGO operator who created the assignment	InspectionSchedule
	Record completion or cancellation events with associated timestamps	
	Support re-assignment to a different actor when the original assignee is unavailable	

2.1.23 SafetyFlag

Class	Responsibilities	Collaborators
SafetyFlag(Entity)	Represent a flag raised against a Docker resource for a safety or policy violation	SafetyFlagController
	Store flag identifier, hazard level (Low/Medium/Critical), violation description, and flagging inspector	UnsafeFoodController
	Maintain the current flag status (Pending Review, Approved, Dismissed)	InspectionReport
	Provide flag data to SafetyFlagController for NGO notification and unsafe food workflow	FoodDonation
	Link to InspectionReport for full inspection context	

	Support photo evidence attachment for visual proof of the violation	
--	---	--

2.1.24 DisposalRecord

Class	Responsibilities	Collaborators
DisposalRecord(Entity)	Record the disposal of a flagged, unsafe Docker resource	DisposalController
	Store disposal method (Compost/Destroy), date, responsible officer, and resource reference	UnsafeFoodController
	Maintain a confirmation status (Arranged, Completed, Failed)	SafetyFlag
	Provide disposal data to DisposalController for record keeping and reporting	
	Link to parent SafetyFlag entity for full safety event chain	
	Support administrative review of all disposal events	

2.1.25 NGO

Class	Responsibilities	Collaborators
NGO(Entity)	Store NGO identifier, name, zone of operation, and accepted resource types	MatchingController
	Represent the organisational entity that accepts and manages donated resources	DonationController
	Maintain a registry of associated Volunteers, Delivery Partners, and Inspectors	InspectorAssignmentController
	Provide NGO data to MatchingController for donation-to-NGO matching	PenaltyController
	Support acceptance and rejection decisions on inspected donations	
	Record penalty actions issued to Donors under this NGO's authority	

2.1.26 MatchingCriteria

Class	Responsibilities	Collaborators
MatchingCriteria(Entity)	Store criteria parameters for donation matching (food type, quantity range, proximity radius, expiry threshold)	MatchingController

	Provide criteria data to MatchingController and RecommendationEngine for scoring	RecommendationEngine
	Support override of individual criteria by the NGO operator	NGO
	Record historical matching decisions linked to this criteria configuration	
	Validate that criteria values fall within acceptable system-defined bounds	
	Enable broadening of criteria when no suitable match is initially found	

2.1.27 IssueReport

Class	Responsibilities	Collaborators
IssueReport(Entity)	Store issue identifier, category, description, location, and timestamp	IssueController
	Represent a problem reported by a Volunteer during an active pickup task	PickupTask
	Maintain a reference to the parent PickupTask and the reporting Volunteer	Volunteer
	Provide issue data to IssueController for NGO notification and corrective action	
	Support photo evidence attachment to substantiate the reported issue	
	Record resolution status and the corrective action taken by the NGO	

2.1.28 PickupTask

Class	Responsibilities	Collaborators
PickupTask(Entity)	Store pickup task identifier, assigned volunteer, pickup location, and time slot	PickupTaskController
	Represent a formal assignment for a Volunteer to collect a food/resource donation	PickupStatusController
	Maintain task status (Assigned, En Route, Arrived, Picked Up, Completed)	IssueController
	Provide task data to PickupTaskController for status update processing	Volunteer
	Track any IssueReports raised during the execution of this task	FoodDonation
	Link to parent FoodDonation for full donation lifecycle continuity	

2.1.29 AvailabilitySchedule

Class	Responsibilities	Collaborators
AvailabilitySchedule(Entity)	Store a Volunteer's available days, time slots, and blocked periods	AvailabilityController
	Provide availability data to AvailabilityController for task assignment eligibility	PickupSchedulerController
	Maintain version history to track schedule changes over time	Volunteer
	Support conflict detection when a new task overlaps an unavailable slot	
	Record the last-updated timestamp for freshness validation	
	Enable temporary marking of all slots as unavailable	

2.1.30 PickupStatus

Class	Responsibilities	Collaborators
PickupStatus(Entity)	Store the current status of an active PickupTask (Assigned/En Route/Arrived/Picked Up)	PickupStatusController
	Record the timestamp and geolocation at each status transition	PickupTask
	Provide status data to PickupStatusController for lifecycle update processing	PickupStatusUpdateUI
	Notify NGO and Donor entities upon each status change event	
	Support queuing and retry of status updates when network connectivity is lost	
	Link to parent PickupTask for full task lifecycle traceability	

2.1.31 DeliveryTask

Class	Responsibilities	Collaborators
DeliveryTask(Entity)	Store delivery task identifier, assigned delivery partner, pickup location, and drop-off destination	DeliveryTaskController
	Represent a formal assignment for a Delivery Partner to transport a food donation	DeliveryStatusController

	Maintain task status (Assigned, Accepted, In Transit, Delayed, Delivered)	DeliveryPartner
	Provide task data to DeliveryTaskController for status update processing	FoodDonation
	Support task acceptance or rejection by the assigned Delivery Partner	
	Link to parent FoodDonation for full donation lifecycle continuity	

2.1.32 DeliveryStatus

Class	Responsibilities	Collaborators
DeliveryStatus(Entity)	Store the current delivery status (In Transit/Delayed/Delivered) with timestamp	DeliveryStatusController
	Record delay reason and estimated extra time when status is Delayed	DeliveryTask
	Provide status data to DeliveryStatusController for real-time update broadcasting	DeliveryStatusUpdateUI
	Notify NGO and Donor upon delivery completion or delay	
	Support status update queuing and retry on connectivity failure	
	Link to parent DeliveryTask for full task lifecycle auditability	

2.1.33 Role

Class	Responsibilities	Collaborators
Role(Entity)	Store the role name (Developer, System Administrator, Inspector, NGO, Volunteer, Delivery Partner)	UserManagementController
	Define the set of permissions granted to users holding this role	Permission
	Provide role data to UserManagementController for access control enforcement	User
	Support role assignment and removal by the System Administrator	
	Maintain a mapping of role to allowed system modules and operations	
	Record role change history for audit trail purposes	

2.1.34 Permission

Class	Responsibilities	Collaborators
Permission(Entity)	Store permission identifier, module name, and allowed operation (Read/Write/Execute/Admin)	UserManagementController
	Represent an atomic access right within the system	Role
	Provide permission data to UserManagementController for RBAC enforcement	
	Support assignment of permissions to Roles	
	Maintain a record of which Roles hold each Permission	
	Enable removal of permissions when a Role's scope is reduced	

2.1.35 ActivityLog

Class	Responsibilities	Collaborators
ActivityLog(Entity)	Store log entry identifier, event type, actor, resource reference, and timestamp	SystemMonitoringController
	Record all system events including login attempts, resource changes, and errors	UserManagementController
	Provide log data to SystemMonitoringController for anomaly detection	ActivityMonitoringUI
	Support filtering of log entries by user, role, event type, or time range	
	Maintain tamper-resistant, append-only storage for audit integrity	
	Enable export of log data for external security review	

2.1.36 Report

Class	Responsibilities	Collaborators
Report(Entity)	Store report identifier, category (Impact/Waste/Logistics), date range, and generated content	ReportController
	Represent a compiled operational report generated from lifecycle and activity data	TrackingController

	Provide report data to ReportController for rendering and export	DonationRecord
	Support export formats including PDF, CSV, and XLS	DistributionRecord
	Maintain a creation timestamp and the Administrator who requested the report	
	Link to DonationRecord and DistributionRecord for data sourcing	

2.1.37 DonationRecord

Class	Responsibilities	Collaborators
DonationRecord(Entity)	Store a summary record of a completed donation including resource type, quantity, and lifecycle timeline	ReportController
	Maintain references to donor, NGO, volunteers, and delivery partner involved	TrackingController
	Provide record data to ReportController for report generation	FoodDonation
	Support query by date range, donor, NGO, or lifecycle status	
	Archive completed donation records for long-term retention	
	Link to parent FoodDonation for full event chain access	

2.1.38 DatabaseRecord

Class	Responsibilities	Collaborators
DatabaseRecord(Entity)	Represent any persistent data record stored in the system database	DatabaseController
	Store record type, creation timestamp, last-modified timestamp, and owning module	BackupController
	Provide record data to DatabaseController for archive, deletion, and update operations	BackupLog
	Support conflict detection when a deletion is attempted on an in-use record	AuditLog
	Maintain linkage to BackupLog for recovery chain traceability	
	Enable integrity check operations that detect and flag inconsistencies	

2.1.39 BackupLog

Class	Responsibilities	Collaborators
BackupLog(Entity)	Record details of each backup operation including timestamp, scope, and outcome	BackupController
	Store backup file location and verification checksum	DatabaseController
	Provide backup data to BackupController for scheduled and on-demand backup management	DatabaseRecord
	Support restoration operations by providing the last successful backup reference	
	Maintain a history of backup events for administrative review	
	Alert DatabaseController when a backup has not been completed within the expected window	

2.1.40 AuditLog

Class	Responsibilities	Collaborators
AuditLog(Entity)	Record all database-modifying operations performed by administrators	DatabaseController
	Store administrator identifier, operation type, record affected, and timestamp	SystemMonitoringController
	Provide audit data to DatabaseController for compliance reporting	ActivityLog
	Maintain read-only, tamper-resistant storage for regulatory auditability	
	Support export of audit entries for external compliance review	
	Flag high-risk operations (bulk deletions, role changes) for special review	

2.1.41 Notification

Class	Responsibilities	Collaborators
Notification(Entity)	Store notification identifier, recipient User, event type, message, and delivery timestamp	NotificationController
	Represent a system-generated alert triggered by a lifecycle or administrative event	AlertDispatcher
	Provide notification data to NotificationController for dispatch and retry management	User

	Track delivery status (Pending/Delivered/Failed) for each notification instance	
	Support in-app and email delivery channel preferences per User	
	Maintain a delivery failure log for retry and administrative review	

2.1.42 TrackingLog

Class	Responsibilities	Collaborators
TrackingLog(Entity)	Record each lifecycle stage transition event for a FoodDonation resource	TrackingController
	Store the stage name, transition timestamp, and responsible actor for each event	DonationTrackingUI
	Provide tracking data to TrackingController for full lifecycle timeline assembly	FoodDonation
	Support query by donation identifier for end-to-end lifecycle visualisation	StatusHistory
	Maintain append-only, tamper-resistant records for audit integrity	
	Enable export of the full lifecycle timeline as a PDF report	

2.1.43 StatusHistory

Class	Responsibilities	Collaborators
StatusHistory(Entity)	Store the complete sequence of status values a resource has held over its lifecycle	TrackingController
	Maintain entry and exit timestamps for each status period	TrackingLog
	Provide status history data to TrackingController for lifecycle report generation	DonationStatus
	Support rollback detection by identifying unexpected state reversals	
	Link to parent TrackingLog for contextual lifecycle continuity	
	Enable filtering of history by status type or time window	

2.1.44 InspectionReport

Class	Responsibilities	Collaborators
InspectionReport(Entity)	Store inspection identifier, outcome (Safe/Unsafe/Requires Review), detailed notes, and inspector reference	InspectionController
	Represent the formal record of a completed food safety or compliance inspection	SafetyFlagController
	Provide report data to InspectionController for NGO decision support	FoodDonation
	Maintain a timestamp and photographic evidence reference for the inspection	FoodInspector
	Link to parent FoodDonation for full lifecycle integration	
	Serve as the triggering document for SafetyFlag creation on unsafe findings	

2.1.45 FoodSafetyRule

Class	Responsibilities	Collaborators
FoodSafetyRule(Entity)	Store rule identifier, category, parameter values (max temperature, min shelf life), and status	FoodSafetyRuleController
	Represent a formal safety or compliance rule governing how donations are assessed	InspectionController
	Provide rule data to FoodSafetyRuleController for publication and enforcement	FoodSafetyGuideline
	Support deactivation of outdated rules without permanent deletion	
	Maintain version history of rule changes with inspector attribution	
	Govern the assessment criteria used during Inspect Food Donation operations	

2.2 Boundary Objects

2.2.1 RegistrationFormUI

Class	Responsibilities	Collaborators
RegistrationFormUI (Boundary)	Render the registration form for new user account creation	RegistrationControllerController
	Collect and validate user input (name, email, password, role selection)	User

	Submit validated registration data to RegistrationController	
	Display inline field-level validation errors to the user	
	Show confirmation message upon successful account creation	
	Support navigation to the Login form	

2.2.2 LoginFormUI

Class	Responsibilities	Collaborators
LoginFormUI(Boundary)	Render the login interface with username and password fields	LoginControllerController
	Capture and submit user credentials to AuthenticationController	User
	Display error messages for invalid credentials or locked accounts	
	Provide a Forgot Password link that triggers the password reset flow	
	Redirect the user to their role-specific dashboard upon successful authentication	
	Mask password field input for security	

2.2.3 AddDonationFormUI

Class	Responsibilities	Collaborators
AddDonationFormUI(Boundary)	Render the Add Donation form with food type, quantity, expiry, and location fields	AddDonationControllerController
	Collect and validate donation details before submission	User
	Submit validated donation data to DonationController	
	Display field-level validation errors (e.g., missing expiry date)	
	Show confirmation message with new donation ID on success	
	Support pre-submission preview of entered details	

2.2.4 DonationListUI

Class	Responsibilities	Collaborators
-------	------------------	---------------

DonationListUI(Boundary)	Display the list of available, pending, or completed donations to the User	DonationListController
	Provide filter controls for food type, distance, and date range	User
	Render each donation entry with summary attributes (type, quantity, location)	
	Support tap-to-expand for full donation detail view	
	Refresh the list in real time as new donations are submitted	
	Show a 'No more donations' message when the list is exhausted	

2.2.5 EditDonationFormUI

Class	Responsibilities	Collaborators
EditDonationFormUI(Boundary)	Render the edit form pre-populated with the existing donation details	EditDonationControllerController
	Allow the User to modify food type, quantity, expiry, and location fields	User
	Submit modified data to DonationManagementController for persistence	
	Display validation errors if the updated data is invalid	
	Show a confirmation message upon successful update	
	Prevent editing of donations that have already been accepted	

2.2.6 DonationStatusDashboardUI

Class	Responsibilities	Collaborators
DonationStatusDashboardUI(Boundary)	Display the real-time lifecycle status of a selected donation	DonationStatusControllerController
	Render each lifecycle stage with a timestamp and responsible actor label	User
	Highlight the current active stage and completed stages visually	
	Provide navigation to the Feedback form upon donation completion	

	Support filtering of donations by status type on the dashboard	
	Show a delay flag indicator when lifecycle progression has stalled	

2.2.7 FeedbackFormUI

Class	Responsibilities	Collaborators
FeedbackFormUI(Boundary)	Render the feedback submission form post donation completion	FeedbackControllerController
	Collect written comments and star rating from the User	User
	Submit feedback data to FeedbackController and rating to RatingController	
	Display a validation error when the rating field is left empty	
	Show a Thank You confirmation message on successful submission	
	Support optional skip of feedback submission	

2.2.8 RatingUI

Class	Responsibilities	Collaborators
RatingUI(Boundary)	Render the interactive star-rating widget (1–5 stars)	RatingController
	Capture the selected rating value from the User	User
	Submit the rating value to RatingController	
	Display current average rating for the resource being reviewed	
	Support half-star precision if required by the system configuration	
	Show a validation prompt if the rating is required but not selected	

2.2.9 GuidelinesViewUI

Class	Responsibilities	Collaborators
GuidelinesViewUI(Boundary)	Display the current food safety guidelines and Docker usage policies	GuidelinesViewController
	Render each policy rule with its category, description, and severity label	User

	Provide an Acknowledge All button that submits confirmation to GuidelineController	
	Support download of the guidelines as a PDF document	
	Display a Verified badge when all guidelines have been acknowledged	
	Show a warning when guidelines are outdated and pending revision	

2.2.10 AvailableDonationsUI

Class	Responsibilities	Collaborators
AvailableDonationsUI(Boundary)	Display the full list of available, uninspected, unaccepted donations to the NGO	AvailableDonationsController
	Provide filter controls for food type, location, and submission date	User
	Render each donation entry with donor name, distance, and quantity	
	Support real-time refresh as new donations are added to the list	
	Show a No Donations Available message when the list is empty	
	Trigger the Donation Matching System view on user request	

2.2.11 DonationDecisionUI

Class	Responsibilities	Collaborators
DonationDecisionUI(Boundary)	Display the full details of a selected donation for NGO review (Accept/Reject)	DonationDecisionController
	Show inspection result outcome (Safe/Unsafe) from InspectionReport	User
	Provide Accept and Reject buttons for the NGO decision	
	Require and capture a rejection reason before allowing rejection submission	
	Submit the decision to DonationController for lifecycle update	
	Display a field completeness indicator for missing donation attributes	

2.2.12 InspectionFormUI

Class	Responsibilities	Collaborators
InspectionFormUI(Boundary)	Render the inspection form for the Food Safety Inspector	InspectionControllerController
	Provide checkboxes for packaging, temperature, and expiry validation	User
	Support Safe/Reject outcome selection with rejection reason input	
	Enable photo evidence upload for the inspection record	
	Submit inspection outcome to InspectionController	
	Display a Generate Report button upon inspection completion	

2.2.13 PickupSchedulingUI

Class	Responsibilities	Collaborators
PickupSchedulingUI(Boundary)	Render the volunteer assignment panel for pickup coordination	PickupSchedulingController
	Display available volunteers with availability status and distance	User
	Show time conflict warnings when a proposed assignment conflicts with an existing slot	
	Provide Confirm Assignment button that triggers TaskAssignmentController	
	Display task details including pickup location and time	
	Support alternative volunteer proposal when the primary volunteer has a conflict	

2.2.14 DistributionManagementUI

Class	Responsibilities	Collaborators
DistributionManagementUI(Boundary)	Display the list of delivery partners available to the NGO	DistributionManagementController
	Show partner name, vehicle type, rating, and zone of operation	User
	Mark unavailable partners as greyed out with a Busy status label	
	Provide a Dispatch Partner button for assignment to a delivery task	

	Support Expand Radius option when no partners are in range	
	Show Confirm Dispatch confirmation message upon assignment	

2.2.15 DonorPenaltyUI

Class	Responsibilities	Collaborators
DonorPenaltyUI(Boundary)	Display the donor oversight panel for penalisation management	DonorPenaltyController
	Show donor name, violation type, and severity classification (Minor/Repeat)	User
	Provide Warn and Penalise buttons for progressive enforcement	
	Display outcome message confirming points deducted and posting restrictions	
	Require violation details to be entered before penalty submission	
	Show historical penalty records for the selected donor	

2.2.16 InspectorAssignmentUI

Class	Responsibilities	Collaborators
InspectorAssignmentUI(Boundary)	Display the inspector management panel for NGO use	InspectorAssignmentController
	List available food safety inspectors with zone and availability status	User
	Provide a Select Inspector dropdown and Assign Inspector button	
	Show a No Inspector Available warning when all inspectors are busy	
	Display confirmation of assignment and inspector notification	
	Support reassignment to a different inspector if the first is unavailable	

2.2.17 UnsafeFoodManagementUI

Class	Responsibilities	Collaborators
UnsafeFoodManagementUI(Boundary)	Display the unsafe food management module for NGO action	UnsafeFoodManagementController

	Show flagged donation details including violation type and hazard level	User
	Provide Compost and Destroy disposal method buttons	
	Support photo evidence upload for disposal documentation	
	Display Generate Disposal Record button upon method selection	
	Show Donation Quarantined confirmation message after action	

2.2.18 MatchingDashboardUI

Class	Responsibilities	Collaborators
MatchingDashboardUI(Boundary)	Display the donation matching dashboard for the NGO	MatchingControllerController
	Show candidate donation details (type, quantity, readiness status)	User
	Provide Min Distance and NGO Preferences filter options	
	List top-matched donations ranked by matching score	
	Show Best Fit label for the highest-scored match	
	Provide Confirm Match button to initiate the acceptance workflow	

2.2.19 IssueReportingUI

Class	Responsibilities	Collaborators
IssueReportingUI(Boundary)	Render the issue reporting form for volunteer use during active pickup	IssueReportingController
	Provide a category dropdown (Donor Absent/Food Unsafe/Vehicle Trouble)	User
	Capture free-text description and optional photo upload	
	Submit report to IssueController on pressing Submit Report	
	Display confirmation of report submission and NGO notification	
	Enforce non-empty description before submission	

2.2.20 AvailabilityFormUI

Class	Responsibilities	Collaborators
AvailabilityFormUI(Boundary)	Render the availability schedule form for volunteers	AvailabilityControllerController
	Provide day-of-week toggle buttons and time slot input fields	User
	Validate time range entries (start must precede end time)	
	Submit availability data to AvailabilityController on Save	
	Display Invalid Time Range error when times are inconsistent	
	Support marking all slots as temporarily unavailable	

2.2.21 PickupRequestsListUI

Class	Responsibilities	Collaborators
PickupRequestsListUI(Boundary)	Display the list of open pickup requests available to the volunteer	PickupRequestsListController
	Show request location name, distance, and time slot for each entry	User
	Support filter by location or food type	
	Display real-time distance updates as the volunteer's location changes	
	Show Tap a Request to View Details prompt at bottom of list	
	Render No Pickup Requests Available when list is empty	

2.2.22 TaskAcceptanceUI

Class	Responsibilities	Collaborators
TaskAcceptanceUI(Boundary)	Display task details for a selected pickup request (location, time, type)	TaskAcceptanceController
	Provide Accept Task and Go Back buttons	User
	Submit task acceptance to PickupTaskController on confirmation	
	Display Task Accepted — Proceed to Pickup confirmation message	

	Show an error if the task has already been accepted by another volunteer	
	Prevent duplicate acceptance of the same task	

2.2.23 PickupStatusUpdateUI

Class	Responsibilities	Collaborators
PickupStatusUpdateUI(Boundary)	Display the active task status tracker with sequential status options	PickupStatusUpdateController
	Render current status (Assigned/En Route/Arrived/Picked Up) with visual indicator	User
	Provide Update Status button to advance to the next stage	
	Support addition of notes and photo attachment to the status update	
	Submit status update to PickupStatusController	
	Notify NGO and Donor on each status change via NotificationUI	

2.2.24 DeliveryTaskUI

Class	Responsibilities	Collaborators
DeliveryTaskUI(Boundary)	Display available delivery tasks for the Delivery Partner	DeliveryTaskController
	Show task identifier, pickup location, drop-off destination, and distance filter	User
	Provide Accept and Skip buttons per task	
	Submit task acceptance to DeliveryTaskController	
	Display Task Accepted — Proceed to Pickup confirmation	
	Show No Tasks Available message when the task queue is empty	

2.2.25 PickupConfirmationUI

Class	Responsibilities	Collaborators
PickupConfirmationUI(Boundary)	Display the pickup confirmation screen for Delivery Partner use	PickupConfirmationController

	Show GPS verification status (Within Range / Out of Range)	User
	Provide Scan QR / Enter ID field for resource identification	
	Show Quantity Match result and Adjust Qty option for discrepancies	
	Capture reason for discrepancy if quantity does not match	
	Submit pickup confirmation to PickupController	

2.2.26 DeliveryConfirmationUI

Class	Responsibilities	Collaborators
DeliveryConfirmationUI(Boundary)	Display the delivery drop-off confirmation screen	DeliveryConfirmationController
	Show recipient availability status (Available / Contact Them buttons)	User
	Provide Sign and Photo proof-of-delivery options	
	Submit delivery confirmation to DeliveryController	
	Display Delivery Successful — Points Awarded message on completion	
	Record timestamp and destination verification on confirmation	

2.2.27 DeliveryStatusUpdateUI

Class	Responsibilities	Collaborators
DeliveryStatusUpdateUI(Boundary)	Display the active delivery status update panel	DeliveryStatusUpdateController
	Show current status (In Transit/Delayed) with selection options	User
	Provide Delay Reason dropdown and Est. Extra Time input when Delayed is selected	
	Submit status update to DeliveryStatusController	
	Display submitted confirmation message	
	Queue updates for retry when network is unavailable	

2.2.28 UserManagementDashboardUI

Class	Responsibilities	Collaborators
UserManagementDashboardUI(Boundary)	Display the user management panel for System Administrator use	UserManagementController
	List all registered users with name, role, and active/inactive status	User
	Provide Change Role and Deactivate buttons per user record	
	Show User Settings Updated confirmation on successful action	
	Support search by name or role for quick user location	
	Display validation error for invalid role assignment data	

2.2.29 ActivityMonitoringUI

Class	Responsibilities	Collaborators
ActivityMonitoringUI(Boundary)	Display the real-time system activity monitoring dashboard	ActivityMonitoringController
	Show live event feed with user actions, resource changes, and errors	User
	Render a Suspicious Spike Detected alert for anomalous activity patterns	
	Provide Filter by Action Type dropdown for event stream narrowing	
	Show Flag Account and Clear buttons for suspicious user accounts	
	Provide Export Audit Logs button for compliance reporting	

2.2.30 ReportGenerationUI

Class	Responsibilities	Collaborators
ReportGenerationUI(Boundary)	Display the report generation module for the Administrator	ReportGenerationController
	Provide report category buttons (Impact/Waste/Logistics)	User
	Accept start date and end date inputs for reporting period selection	

	Trigger report generation on Generate Report button press	
	Provide PDF, CSV, and XLS export format buttons	
	Display no-data warning when the selected period has insufficient records	

2.2.31 DatabaseManagementUI

Class	Responsibilities	Collaborators
DatabaseManagementUI(Boundary)	Display the database console for System Administrator use	DatabaseManagementController
	Show current storage utilisation percentage	User
	Provide Run Backup and Archive buttons for maintenance operations	
	Display Run Integrity Check button with outcome message	
	Show Inconsistency Found — Auto-Repair Running alert on detection	
	Display Database Optimised and Secured confirmation on completion	

2.2.32 NotificationUI

Class	Responsibilities	Collaborators
NotificationUI(Boundary)	Display the notification centre for the current User	NotificationController
	List recent notifications with event type, timestamp, and read status	User
	Show New Donation Available, Task Completed, and Pickup Reminder entries	
	Support tap-to-open for full notification detail	
	Display Failed Alerts Retry via SMS prompt for undelivered notifications	
	Mark notifications as read on open	

2.2.33 DonationTrackingUI

Class	Responsibilities	Collaborators
DonationTrackingUI(Boundary)	Display the donation lifecycle traceability view	DonationTrackingController

	Accept a Donation ID input for direct lookup	User
	Render each lifecycle stage as a timeline entry with timestamp and actor	
	Show a Delay Flagged indicator at the stage where a delay occurred	
	Provide Export Lifecycle PDF button for download	
	Display Donation Record Not Found error for invalid identifiers	

2.2.34 UnsafeFoodFlagUI

Class	Responsibilities	Collaborators
UnsafeFoodFlagUI(Boundary)	Display the safety alert panel for Inspector use	UnsafeFoodFlagController
	Show flagged donation details (batch ID, rejection reason, item count)	User
	Provide Hazard Level selector (Low/Medium/Critical)	
	Display No Photo Attached request when evidence is missing	
	Provide Approve Safety Flag button to confirm the flag	
	Show Donation Quarantined — Food Bank Notified confirmation	

2.3 Control Objects

2.3.1 AuthenticationController

Class	Responsibilities	Collaborators
AuthenticationController(Control)	Validate submitted username and password against stored Credentials entity	User
	Retrieve the Role associated with the authenticated User for dashboard routing	Credentials
	Reject access and log failed attempt when credentials are invalid	Role
	Initiate password reset flow when the Forgot Password action is triggered	LoginFormUI
	Generate and store session token on successful authentication	RegistrationController

	Enforce account lockout after a configurable number of failed login attempts	
--	--	--

2.3.2 RegistrationController

Class	Responsibilities	Collaborators
RegistrationController(Control)	Validate new user registration input for completeness and format correctness	User
	Check for duplicate username or email before creating a new User entity	DonorProfile
	Create User, DonorProfile, and Credentials entities upon successful validation	Credentials
	Assign the appropriate Role to the new user based on selected registration type	Role
	Trigger a confirmation notification to the new User via NotificationController	RegistrationFormUI
	Log the registration event to ActivityLog for security auditing	NotificationController

2.3.3 DonationController

Class	Responsibilities	Collaborators
DonationController(Control)	Process new donation submissions from AddDonationFormUI and persist FoodDonation entity	FoodDonation
	Validate donation data against food safety guidelines before acceptance	DonationValidationController
	Retrieve and return the list of available donations for NGO and volunteer views	DonationManagementController
	Update donation lifecycle status in response to NGO, Inspector, and Volunteer actions	DonationTrackingController
	Trigger NGO notification when a new donation matching their profile is submitted	NotificationController
	Coordinate with DonationTrackingController to record each lifecycle event	AddDonationFormUI

2.3.4 DonationValidationController

Class	Responsibilities	Collaborators
DonationValidationController(Control)	Apply food safety and policy rules to submitted donation data before persistence	FoodDonation
	Check expiry date validity, food type classification, and quantity range	FoodSafetyRule

	Return structured validation errors to AddDonationFormUI for field-level display	DonationController
	Enforce mandatory acknowledgment of FoodSafetyGuideline before donation submission	AddDonationFormUI
	Log all validation failures to ActivityLog for reporting	
	Invoke FoodSafetyRuleController to retrieve current rule set for validation	

2.3.5 DonationManagementController

Class	Responsibilities	Collaborators
DonationManagementController(Control)	Process donation edit and cancellation requests from EditDonationFormUI	FoodDonation
	Enforce business rules preventing edit or cancellation of accepted donations	DonationController
	Persist updated donation data and record change history in DonationRecord	EditDonationFormUI
	Notify relevant stakeholders of cancellation via NotificationController	NotificationController
	Validate that modified data satisfies current safety and policy rules	DonationTrackingController
	Trigger DonationTrackingController to log the management event	

2.3.6 DonationTrackingController

Class	Responsibilities	Collaborators
DonationTrackingController(Control)	Record each lifecycle stage transition of a FoodDonation in TrackingLog	TrackingLog
	Assemble the full lifecycle timeline from TrackingLog and StatusHistory for display	StatusHistory
	Provide lifecycle data to ReportController for report generation	FoodDonation
	Detect and flag lifecycle delays when a stage has not progressed within the expected window	ReportController
	Enforce valid state transition sequence and reject invalid transitions	DonationTrackingUI
	Respond to Administrator queries for full donation lifecycle traceability	

2.3.7 FeedbackController

Class	Responsibilities	Collaborators
FeedbackController (Control)	Receive and persist Feedback entity submitted via FeedbackFormUI	Feedback
	Link the submitted feedback to the relevant completed FoodDonation record	RatingController
	Retrieve all feedback entries for a given donation for NGO performance review	FoodDonation
	Enforce that feedback can only be submitted for donations in Delivered status	FeedbackFormUI
	Coordinate with RatingController to ensure feedback and rating are stored atomically	ReportController
	Provide aggregated feedback data to ReportController	

2.3.8 RatingController

Class	Responsibilities	Collaborators
RatingController (Control)	Receive and persist Rating entity submitted via RatingUI	Rating
	Validate that rating value falls within the defined range (1–5)	FeedbackController
	Compute and update the average rating for the associated FoodDonation resource	FoodDonation
	Retrieve all ratings for a given resource for performance analysis	RatingUI
	Enforce that ratings can only be submitted for completed donations	
	Coordinate with FeedbackController for atomic feedback-rating persistence	

2.3.9 GuidelineController

Class	Responsibilities	Collaborators
GuidelineController (Control)	Retrieve current FoodSafetyGuideline rules from the data store for UI display	FoodSafetyGuideline
	Record User acknowledgment of guidelines and unlock donation submission	FoodSafetyRuleController
	Support guideline PDF export triggered from GuidelinesViewUI	GuidelinesViewUI
	Notify FoodSafetyRuleController when a guideline update is published	ActivityLog

	Enforce that all guidelines must be acknowledged before AddDonationFormUI unlocks	
	Log acknowledgment events to ActivityLog for compliance auditing	

2.3.10 PickupSchedulerController

Class	Responsibilities	Collaborators
PickupSchedulerController(Control)	Retrieve available volunteers and their AvailabilitySchedule for assignment	Volunteer
	Detect time conflicts between proposed task slots and existing volunteer schedules	AvailabilitySchedule
	Propose alternative volunteers when the primary assignee has a conflict	PickupTask
	Persist pickup task assignments to PickupTask entity	TaskAssignmentController
	Notify the assigned Volunteer via NotificationController	NotificationController
	Update Volunteer schedule to block the assigned time slot	PickupSchedulingUI

2.3.11 TaskAssignmentController

Class	Responsibilities	Collaborators
TaskAssignmentController(Control)	Assign pickup tasks to available Volunteers based on proximity and availability	Assignment
	Create and persist Assignment entity records for each task assignment	Volunteer
	Notify the assigned Volunteer and the NGO of the new assignment	PickupTask
	Support re-assignment when the original Volunteer declines or becomes unavailable	FoodDonation
	Update FoodDonation status to Pickup Assigned upon successful assignment	NotificationController
	Enforce that only inspected and accepted donations can have pickup tasks assigned	

2.3.12 DistributionController

Class	Responsibilities	Collaborators
DistributionController(Control)	Manage the list of Delivery Partners and their assignment to active deliveries	DeliveryPartner

	Create and persist DistributionRecord upon delivery task completion	DistributionRecord
	Match available Delivery Partners to pending delivery tasks by zone and availability	DeliveryTask
	Support partner re-assignment when the primary partner declines a task	FoodDonation
	Update FoodDonation status upon successful delivery	ReportController
	Provide distribution data to ReportController for logistics reporting	DistributionManagementUI

2.3.13 PenaltyController

Class	Responsibilities	Collaborators
PenaltyController(Control)	Process penalisation requests from DonorPenaltyUI submitted by NGO administrators	PenaltyRecord
	Create and persist PenaltyRecord linked to the offending Donor entity	ViolationAssessmentController
	Enforce progressive enforcement (Warning before Penalty) based on severity	Donor
	Notify the Donor of the imposed penalty via NotificationController	NotificationController
	Trigger posting restrictions on the Donor account when penalty is issued	AuditLog
	Log penalty actions to AuditLog for administrative accountability	DonorPenaltyUI

2.3.14 ViolationAssessmentController

Class	Responsibilities	Collaborators
ViolationAssessmentController(Control)	Assess the severity of a detected violation based on type, recurrence, and impact	Violation
	Retrieve applicable FoodSafetyRule entries for violation classification	FoodSafetyRule
	Return severity classification (Minor/Major/Critical) to PenaltyController	PenaltyController
	Record the assessment outcome in Violation entity	ActivityLog
	Support escalation recommendation when a violation is classified as Critical	

	Log assessment events to ActivityLog for audit purposes	
--	---	--

2.3.15 InspectorAssignmentController

Class	Responsibilities	Collaborators
InspectorAssignmentController(Control)	Retrieve available FoodInspectors and their InspectionSchedule for assignment	FoodInspector
	Assign an Inspector to a pending donation and create InspectionSchedule entry	InspectionSchedule
	Notify the assigned Inspector via NotificationController	Assignment
	Support reassignment when the primary Inspector is unavailable	NotificationController
	Log assignment actions to ActivityLog	ActivityLog
	Update Assignment entity status upon Inspector assignment confirmation	InspectorAssignmentUI

2.3.16 UnsafeFoodController

Class	Responsibilities	Collaborators
UnsafeFoodController(Control)	Receive SafetyFlag from SafetyFlagController and initiate unsafe food workflow	SafetyFlag
	Remove flagged donation from available listings in the system	DisposalRecord
	Guide NGO through disposal method selection via UnsafeFoodManagementUI	FoodDonation
	Create and persist DisposalRecord upon disposal method confirmation	NotificationController
	Notify relevant stakeholders (NGO, Donor, Admin) of the unsafe food action	UnsafeFoodManagementUI
	Update FoodDonation lifecycle status to Disposed	DisposalController

2.3.17 DisposalController

Class	Responsibilities	Collaborators
DisposalController(Control)	Persist DisposalRecord entities with method, date, officer, and evidence reference	DisposalRecord
	Validate that a disposal reason and method have been provided before persistence	FoodDonation

	Update associated FoodDonation status to reflect completed disposal	AuditLog
	Log disposal event to AuditLog for administrative review	NotificationController
	Notify NGO Administrator upon successful disposal record creation	UnsafeFoodController
	Support administrative query of all disposal records by date or method	

2.3.18 MatchingController

Class	Responsibilities	Collaborators
MatchingController (Control)	Scan all available FoodDonation listings against NGO MatchingCriteria	FoodDonation
	Score candidate donations using RecommendationEngine based on type, quantity, expiry, and proximity	MatchingCriteria
	Return a ranked list of matching donations to MatchingDashboardUI	RecommendationEngine
	Support manual override when NGO selects a non-top-ranked donation	NGO
	Notify the selected Donor when their donation has been matched	NotificationController
	Log matching decisions to ActivityLog for analysis	MatchingDashboardUI

2.3.19 RecommendationEngine

Class	Responsibilities	Collaborators
RecommendationEngine(Control)	Apply multi-factor scoring algorithm to rank candidate FoodDonation entities	MatchingController
	Weight scoring factors according to MatchingCriteria parameters (proximity, quantity, expiry)	MatchingCriteria
	Return ranked score list to MatchingController for top-match selection	FoodDonation
	Support dynamic re-weighting when NGO adjusts preference filters	
	Log scoring computation for transparency and auditability	
	Handle empty candidate set gracefully by returning a broadening recommendation	

2.3.20 IssueController

Class	Responsibilities	Collaborators
IssueController(Control)	Receive issue reports from IssueReportingUI during active pickup tasks	IssueReport
	Persist IssueReport entity with description, category, timestamp, and evidence	PickupTask
	Notify the NGO of the reported issue via NotificationController	NotificationController
	Link the IssueReport to the active PickupTask for traceability	ActivityLog
	Log the issue event to ActivityLog	IssueReportingUI
	Support NGO resolution of the issue with corrective action recording	

2.3.21 AvailabilityController

Class	Responsibilities	Collaborators
AvailabilityController(Control)	Receive and persist AvailabilitySchedule updates submitted via AvailabilityFormUI	AvailabilitySchedule
	Validate that submitted time ranges are coherent (start before end)	Volunteer
	Detect conflicts between new availability entries and existing task assignments	PickupSchedulerController
	Update Volunteer entity availability status based on submitted schedule	ActivityLog
	Provide availability data to PickupSchedulerController for assignment filtering	AvailabilityFormUI
	Log availability changes to ActivityLog	

2.3.22 PickupRequestController

Class	Responsibilities	Collaborators
PickupRequestController(Control)	Retrieve and return open pickup requests for display in PickupRequestsListUI	PickupRequest
	Filter requests by proximity, food type, or time based on Volunteer preferences	Volunteer
	Maintain real-time availability of the pickup request list	NotificationController
	Remove requests from the list when they are accepted by a Volunteer	PickupRequestsListUI

	Notify Volunteers of newly available pickup requests via NotificationController	
	Log request view events for reporting purposes	

2.3.23 PickupTaskController

Class	Responsibilities	Collaborators
PickupTaskController(Control)	Process Volunteer task acceptance from TaskAcceptanceUI	PickupTask
	Assign the PickupTask to the accepting Volunteer and update task status	Volunteer
	Prevent duplicate acceptance of the same task by concurrent Volunteers	FoodDonation
	Notify the Donor and NGO of the task acceptance via NotificationController	NotificationController
	Update FoodDonation status to Pickup Assigned	ActivityLog
	Log task acceptance event to ActivityLog	TaskAcceptanceUI

2.3.24 PickupStatusController

Class	Responsibilities	Collaborators
PickupStatusController(Control)	Receive status updates from PickupStatusUpdateUI during active pickup tasks	PickupStatus
	Persist PickupStatus entity with new status value, timestamp, and geolocation	PickupTask
	Update the parent PickupTask and FoodDonation lifecycle status	FoodDonation
	Notify NGO and Donor of each status change via NotificationController	NotificationController
	Queue status updates for retry when network connectivity is unavailable	TrackingLog
	Log status change events to TrackingLog for lifecycle record	PickupStatusUpdateUI

2.3.25 DeliveryTaskController

Class	Responsibilities	Collaborators
DeliveryTaskController(Control)	Process Delivery Partner task acceptance from DeliveryTaskUI	DeliveryTask

	Assign the DeliveryTask to the accepting partner and update task status	DeliveryPartner
	Prevent duplicate acceptance by concurrent delivery partners	FoodDonation
	Notify the NGO of task acceptance via NotificationController	NotificationController
	Update FoodDonation status to Delivery Accepted	ActivityLog
	Log task acceptance event to ActivityLog	DeliveryTaskUI

2.3.26 PickupController

Class	Responsibilities	Collaborators
PickupController(Control)	Process physical pickup confirmation from PickupConfirmationUI	FoodDonation
	Verify GPS location of the Delivery Partner against expected pickup coordinates	DistributionRecord
	Validate resource identifier via QR scan or manual ID entry	DeliveryPartner
	Record quantity match or discrepancy in DistributionRecord	NotificationController
	Update FoodDonation lifecycle status to Food Picked Up	PickupConfirmationUI
	Notify the NGO of confirmed pickup via NotificationController	

2.3.27 DeliveryController

Class	Responsibilities	Collaborators
DeliveryController(Control)	Process delivery confirmation from DeliveryConfirmationUI	FoodDonation
	Verify recipient availability and record proof of delivery (signature or photo)	DistributionRecord
	Create and persist DistributionRecord upon successful delivery	Beneficiary
	Update FoodDonation lifecycle status to Delivered	NotificationController
	Notify the NGO, Donor, and Beneficiary of successful delivery	DeliveryConfirmationUI
	Trigger feedback and rating prompt for the Donor via NotificationController	

2.3.28 DeliveryStatusController

Class	Responsibilities	Collaborators
DeliveryStatusController(Control)	Receive status updates from DeliveryStatusUpdateUI during active deliveries	DeliveryStatus
	Persist DeliveryStatus entity with status value, delay reason, and timestamp	DeliveryTask
	Update the parent DeliveryTask and FoodDonation lifecycle status	FoodDonation
	Notify the NGO of status changes, particularly delay events	NotificationController
	Queue status updates for retry when network connectivity is lost	TrackingLog
	Log status events to TrackingLog for full lifecycle continuity	DeliveryStatusUpdateUI

2.3.29 UserManagementController

Class	Responsibilities	Collaborators
UserManagementController(Control)	Create, update, deactivate, and delete User accounts on Administrator instruction	User
	Assign and revoke Roles and Permissions per user account	Role
	Enforce role-based access control policies on all system module access	Permission
	Search user accounts by name or role for administrative efficiency	AuditLog
	Log all user management actions to AuditLog	NotificationController
	Notify Users of account changes via NotificationController	UserManagementDashboard UI

2.3.30 SystemMonitoringController

Class	Responsibilities	Collaborators
SystemMonitoringController(Control)	Retrieve real-time ActivityLog entries for display in ActivityMonitoringUI	ActivityLog
	Detect anomalous activity patterns (e.g., repeated failed logins) and raise alerts	AuditLog
	Flag suspicious user accounts for Administrator review	NotificationController
	Support filtering of event streams by user, role, event type, or time range	ActivityMonitoringUI

	Provide export of audit log data to AuditLog for compliance reporting	
	Log monitoring actions and alert events to ActivityLog	

2.3.31 ReportController

Class	Responsibilities	Collaborators
ReportController(Control)	Generate operational reports based on date range, category, and user parameters	Report
	Retrieve donation lifecycle data from DonationRecord and TrackingLog	DonationRecord
	Retrieve distribution data from DistributionRecord for logistics reporting	DistributionRecord
	Compile and render the Report entity for UI display	TrackingController
	Support export of generated reports in PDF, CSV, and XLS formats	ReportGenerationUI
	Enforce that reports can only be generated when sufficient data exists	

2.3.32 DatabaseController

Class	Responsibilities	Collaborators
DatabaseController(Control)	Execute archive, deletion, and update operations on DatabaseRecord entities	DatabaseRecord
	Block deletion of records that are actively referenced by other entities	BackupController
	Trigger backup operations via BackupController on schedule or on demand	AuditLog
	Run integrity checks on the database and invoke auto-repair when inconsistencies are found	DatabaseManagementUI
	Log all database operations to AuditLog with administrator ID and timestamp	
	Display storage utilisation and operation outcomes in DatabaseManagementUI	

2.3.33 BackupController

Class	Responsibilities	Collaborators
BackupController(Control)	Execute scheduled and on-demand backup operations for all DatabaseRecord entities	BackupLog

	Create BackupLog entries recording backup scope, timestamp, and verification checksum	DatabaseController
	Alert DatabaseController when a backup has not completed within the expected window	AuditLog
	Support restoration by providing the most recent successful BackupLog reference	DatabaseManagementUI
	Validate backup integrity via checksum verification	
	Log backup events to AuditLog for administrative accountability	

2.3.34 NotificationController

Class	Responsibilities	Collaborators
NotificationController(Control)	Dispatch in-app and email notifications upon system-triggered events	Notification
	Create and persist Notification entities for each dispatched alert	AlertDispatcher
	Retry delivery when the initial dispatch fails, logging each attempt	User
	Deliver notifications within 3 seconds of the triggering event	NotificationUI
	Allow Administrators to configure custom notification rules	
	Mark notifications as read when opened by the recipient User	

2.3.35 AlertDispatcher

Class	Responsibilities	Collaborators
AlertDispatcher(Control)	Receive notification dispatch requests from NotificationController	NotificationController
	Route notifications to the correct delivery channel (in-app or email) based on User preferences	Notification
	Execute the actual message delivery to the recipient User	User
	Return delivery confirmation or failure status to NotificationController	
	Queue failed notifications for retry via SMS when in-app and email both fail	

	Log all dispatch events including channel, timestamp, and outcome	
--	---	--

2.3.36 TrackingController

Class	Responsibilities	Collaborators
TrackingController(Control)	Record lifecycle stage transitions to TrackingLog on instruction from domain controllers	TrackingLog
	Assemble the full lifecycle timeline from TrackingLog and StatusHistory for display	StatusHistory
	Provide lifecycle data to ReportController for report generation	FoodDonation
	Detect lifecycle delays and flag them in TrackingLog entries	ReportController
	Support Administrator queries for donation traceability by donation ID	DonationTrackingUI
	Enable export of the lifecycle timeline as a PDF report	

2.3.37 InspectionController

Class	Responsibilities	Collaborators
InspectionController(Control)	Receive inspection outcomes from InspectionFormUI and persist InspectionReport	InspectionReport
	Validate that all required inspection fields (packaging, temperature, expiry) are completed	FoodSafetyRule
	Notify the NGO of the inspection result via NotificationController	SafetyFlagController
	Trigger SafetyFlagController when the inspection outcome is Unsafe	NotificationController
	Update FoodDonation lifecycle status to Inspected	FoodDonation
	Retrieve current FoodSafetyRule entries to guide the inspection assessment	InspectionFormUI

2.3.38 FoodSafetyRuleController

Class	Responsibilities	Collaborators
FoodSafetyRuleController(Control)	Create, update, and deactivate FoodSafetyRule entities on Inspector instruction	FoodSafetyRule

	Validate that new rules do not conflict with existing active rules	GuidelineController
	Publish updated rules to GuidelineController and InspectionController	InspectionController
	Log rule changes to AuditLog with inspector attribution and timestamp	AuditLog
	Support deactivation of outdated rules without deletion	DonationValidationController
	Provide the current active rule set to DonationValidationController	

2.3.39 SafetyFlagController

Class	Responsibilities	Collaborators
SafetyFlagController(Control)	Receive and persist SafetyFlag entities from the Inspector via UnsafeFoodFlagUI	SafetyFlag
	Validate that a hazard level, violation description, and inspector reference are provided	InspectionReport
	Notify the NGO of the new safety flag via NotificationController	NotificationController
	Trigger UnsafeFoodController to initiate the unsafe food disposal workflow	UnsafeFoodController
	Link the SafetyFlag to the corresponding InspectionReport for audit chain	UnsafeFoodFlagUI
	Reject flagging submissions that lack a required violation description	

3. Identifying Associations

The following table defines the primary associations between the objects in the Docker tool system. Each association describes a meaningful structural or behavioural relationship, its direction, and its multiplicity.

Source Class	Relationship	Target Class	Multiplicity
User	logs in via	Credentials	1 to 1
User	has a	Role	1 to 1
Role	grants	Permission	1 to many
User	has profile	DonorProfile	1 to 0..1

Donor	submits	FoodDonation	1 to many
FoodDonation	has current	DonationStatus	1 to 1
FoodDonation	is made of	FoodItem (ImageLayer)	1 to many
FoodDonation	is located at	Location	1 to 1
FoodDonation	has	InspectionReport	1 to 0..1
InspectionReport	may produce	SafetyFlag	1 to 0..1
SafetyFlag	triggers creation of	DisposalRecord	1 to 0..1
SafetyFlag	may lead to	PenaltyRecord	0..1 to 1
PenaltyRecord	is linked to	Violation	1 to 1
Violation	is detected in	FoodDonation	many to 1
FoodInspector	conducts	InspectionReport	1 to many
FoodInspector	holds	InspectionSchedule	1 to many
InspectionSchedule	is formalised as	Assignment	1 to 1
NGO	manages	Volunteer	1 to many
NGO	manages	DeliveryPartner	1 to many
NGO	assigns	FoodInspector	1 to many
NGO	uses	MatchingCriteria	1 to 1
Volunteer	has	AvailabilitySchedule	1 to 1
Volunteer	accepts	PickupTask	1 to many
PickupTask	references	FoodDonation	many to 1
PickupTask	has	PickupStatus	1 to many
PickupTask	may generate	IssueReport	1 to many
DeliveryPartner	accepts	DeliveryTask	1 to many
DeliveryTask	references	FoodDonation	many to 1
DeliveryTask	has	DeliveryStatus	1 to many
FoodDonation	is tracked by	TrackingLog	1 to many
TrackingLog	maintains	StatusHistory	1 to many
FoodDonation	produces	DonationRecord	1 to 1
DonationRecord	is included in	Report	many to many
DistributionRecord	delivers to	Beneficiary	many to 1
FoodDonation	receives	Feedback	1 to many

FoodDonation	receives	Rating	1 to many
FoodSafetyRule	governs	InspectionReport	many to many
FoodSafetyGuideline	is based on	FoodSafetyRule	many to 1
Notification	is sent to	User	many to 1
ActivityLog	records actions of	User	many to 1
AuditLog	records operations on	DatabaseRecord	many to 1
BackupLog	is managed by	DatabaseRecord	many to many
Schedule	belongs to	Volunteer	1 to 1
PickupRequest	is fulfilled by	PickupTask	1 to 0..1
RecommendationEngine	scores	FoodDonation	1 to many

3.1 Association Explanation

The associations in this system reflect the full lifecycle of a Docker resource (modelled as FoodDonation) from initial submission through inspection, acceptance, assignment, pickup, delivery, and final reporting. Key structural relationships are:

- The User entity is the root actor for all operations. Every action in the system is ultimately initiated by a User who holds a specific Role.
- The FoodDonation entity is the central domain object. It participates in associations with DonationStatus, InspectionReport, PickupTask, DeliveryTask, TrackingLog, and Feedback, reflecting its pivotal role in the lifecycle.
- SafetyFlag, PenaltyRecord, and Violation form a safety enforcement chain. A flag raised by an Inspector may lead to a penalty record being issued against the offending Donor.
- PickupTask and DeliveryTask represent the two halves of the logistics chain. Each references the parent FoodDonation and maintains its own status history.
- TrackingLog and StatusHistory together provide a complete, tamper-resistant audit trail of every lifecycle stage the FoodDonation has passed through.

4. Identifying Aggregates

Aggregation models whole-part relationships. Two strengths of aggregation are distinguished: Composition (strong, lifecycle-dependent) and Aggregation (weak, independent lifecycle).

4.1 Composition Relationships (Strong)

In Composition, the part cannot exist independently of the whole. If the whole is destroyed, the parts are also destroyed.

Whole	Part	Justification
FoodDonation	FoodItem (ImageLayer)	Each FoodItem is a layer that exclusively belongs to one FoodDonation (Image). Deleting the Image removes all its layers.
FoodDonation	DonationStatus	DonationStatus is a state record that has no meaning outside the context of its parent FoodDonation. It is created and destroyed with the donation.
PickupTask	PickupStatus	PickupStatus entries are lifecycle records of a specific PickupTask. They cannot exist without the task that they describe.
DeliveryTask	DeliveryStatus	DeliveryStatus entries describe the progress of a specific DeliveryTask. They are created as part of that task and have no independent existence.
FoodDonation	TrackingLog	The TrackingLog entries are specific to one FoodDonation's lifecycle. They are created when the donation progresses through stages and are deleted with it.
TrackingLog	StatusHistory	StatusHistory records are sub-entries within a TrackingLog. They are created to detail the time spent in each status and are meaningless without the parent log.
InspectionReport	SafetyFlag	A SafetyFlag is raised as a direct result of an InspectionReport finding. Without the inspection, no flag can exist.
DatabaseRecord	AuditLog	Each AuditLog entry is a record of an operation performed on a DatabaseRecord. The entry is meaningless without the originating record context.

4.2 Aggregation Relationships (Weak)

In Aggregation, the part can exist independently of the whole. The whole references the part, but the part has its own lifecycle.

Whole	Part	Justification
NGO	Volunteer	A Volunteer has an independent account in the system. The NGO manages a roster of Volunteers but removing the NGO does not delete the Volunteer records.
NGO	DeliveryPartner	Similarly, DeliveryPartner entities are independent users. The NGO aggregates them

		for its operations but does not own their lifecycle.
NGO	FoodInspector	FoodInspector entities are independent system actors. An NGO assigns them to donations but they exist independently.
FoodDonation	PickupTask	A PickupTask is created in association with a FoodDonation but references the donation rather than being contained within it. Multiple tasks could theoretically reference the same donation.
FoodDonation	DeliveryTask	As above; DeliveryTask entities reference FoodDonation but have their own independent entity lifecycle managed by the delivery subsystem.
Volunteer	AvailabilitySchedule	The AvailabilitySchedule is logically part of the Volunteer's profile but is managed as an independent entity to support versioning and history tracking.
Volunteer	Schedule	A Schedule object represents time-slot data that is managed independently and can be updated without affecting the Volunteer entity itself.
FoodDonation	Feedback	Feedback is submitted by a User after a donation is complete. It references the donation but exists as a standalone review record.
FoodDonation	Rating	Rating follows the same logic as Feedback; it is submitted independently and linked to the donation by reference.
DatabaseRecord	BackupLog	BackupLog records reference the DatabaseRecord scope but are managed independently by the backup subsystem.

5. Identifying Attributes

The following section lists the principal attributes for each entity object. Visibility is indicated as: + (public), - (private), # (protected).

User

- - userID : String
- - username : String
- - passwordHash : String
- - email : String
- # role : Role
- - createdAt : DateTime
- - lastLoginAt : DateTime

- + isActive : Boolean

DonorProfile (DockerUser)

- - profileID : String
- - displayName : String
- - organisation : String
- - contactNumber : String
- # userRef : User
- - dockerContext : String
- + registryCredentials : String

Credentials

- - credentialID : String
- - hashedPassword : String
- - authToken : String
- - tokenExpiry : DateTime
- - failedAttempts : Integer
- - isLocked : Boolean

FoodDonation (DockerResource)

- - donationID : String
- - resourceType : String
- - name : String
- # status : DonationStatus
- - createdAt : DateTime
- - submittedBy : Donor
- + quantity : Float
- - expiryDate : Date
- - pickupLocation : Location

FoodItem (ImageLayer)

- - layerID : String
- - digest : String
- - sizeBytes : Long
- - position : Integer
- - buildInstruction : String
- - isCached : Boolean

Donor

- - donorID : String
- - name : String
- - email : String
- + penaltyCount : Integer
- - isRestricted : Boolean
- - registeredAt : DateTime

Location

- - locationID : String
- - address : String
- - city : String
- - zone : String
- - latitude : Double
- - longitude : Double
- + ipRange : String

DonationStatus

- - statusID : String
- + currentState : String
- - enteredAt : DateTime
- - triggeredBy : String
- - previousState : String

Feedback

- - feedbackID : String
- - commentText : String
- - submittedAt : DateTime
- # donationRef : FoodDonation
- # authorRef : User

Rating

- - ratingID : String
- + score : Integer
- - submittedAt : DateTime
- # donationRef : FoodDonation
- # raterRef : User

FoodSafetyGuideline (DockerPolicy)

- - guidelineID : String

- - title : String
- - description : String
- + severity : String
- - effectiveDate : Date
- - isActive : Boolean
- - version : Integer

PickupRequest (TaskRequest)

- - requestID : String
- - resourceType : String
- - pickupLocation : Location
- - requiredTimeSlot : DateTime
- + assignmentStatus : String
- # assignedVolunteer : Volunteer

Schedule

- - scheduleID : String
- - availableDays : String[]
- - startTime : Time
- - endTime : Time
- - lastUpdated : DateTime
- - isBlocked : Boolean

Volunteer

- - volunteerID : String
- - name : String
- - contactNumber : String
- + availabilityStatus : String
- - averageRating : Float
- - tasksCompleted : Integer

DeliveryPartner

- - partnerID : String
- - name : String
- - vehicleType : String
- - zone : String
- + isAvailable : Boolean
- - averageRating : Float

DistributionRecord

- - recordID : String
- - resourceType : String
- - quantity : Float
- - sourceLocation : String
- - destinationLocation : String
- - completedAt : DateTime
- # partnerRef : DeliveryPartner

Beneficiary

- - beneficiaryID : String
- - name : String
- - address : String
- - resourceRequirements : String
- + deliveryReceiptCount : Integer

PenaltyRecord

- - penaltyID : String
- - penaltyType : String
- - severity : String
- - issuedDate : Date
- # donorRef : Donor
- # violationRef : Violation
- + isActive : Boolean

Violation

- - violationID : String
- - type : String
- - description : String
- - detectedDate : Date
- + isResolved : Boolean
- # donationRef : FoodDonation

FoodInspector (DockerInspector)

- - inspectorID : String
- - name : String
- - zone : String
- + availabilityStatus : String
- - inspectionsCompleted : Integer

InspectionSchedule

- - scheduleID : String
- - scheduledDate : Date
- - scheduledTime : Time
- + status : String
- # inspectorRef : FoodInspector
- # donationRef : FoodDonation

Assignment

- - assignmentID : String
- - type : String
- + status : String
- - createdAt : DateTime
- - completedAt : DateTime
- # assignedActor : String

SafetyFlag

- - flagID : String
- + hazardLevel : String
- - violationDescription : String
- - flaggedAt : DateTime
- - photoEvidenceRef : String
- + status : String
- # inspectorRef : FoodInspector

DisposalRecord

- - disposalID : String
- - method : String
- - disposalDate : Date
- - responsibleOfficer : String
- + confirmationStatus : String
- # flagRef : SafetyFlag

NGO

- - ngoID : String
- - name : String
- - zone : String
- + acceptedResourceTypes : String[]

- - registeredAt : DateTime
- - isActive : Boolean

MatchingCriteria

- - criteriaID : String
- - foodType : String
- - minQuantity : Float
- - maxProximityKm : Float
- - minShelfLifeDays : Integer
- + lastUpdated : DateTime

IssueReport

- - issueID : String
- - category : String
- - description : String
- - reportedAt : DateTime
- - photoEvidenceRef : String
- + resolutionStatus : String
- # taskRef : PickupTask

PickupTask

- - taskID : String
- - pickupLocation : Location
- - assignedTimeSlot : DateTime
- + taskStatus : String
- # volunteerRef : Volunteer
- # donationRef : FoodDonation

AvailabilitySchedule

- - scheduleID : String
- - availableDays : String[]
- - timeSlots : String[]
- - isTemporarilyUnavailable : Boolean
- - lastUpdated : DateTime

PickupStatus

- - statusID : String
- + currentStatus : String
- - recordedAt : DateTime

- - geolocation : String
- # taskRef : PickupTask

DeliveryTask

- - taskID : String
- - pickupLocation : Location
- - dropOffLocation : Location
- + taskStatus : String
- # partnerRef : DeliveryPartner
- # donationRef : FoodDonation

DeliveryStatus

- - statusID : String
- + currentStatus : String
- - delayReason : String
- - estimatedExtraMinutes : Integer
- - recordedAt : DateTime
- # taskRef : DeliveryTask

Role

- - roleID : String
- + roleName : String
- - description : String
- # permissions : Permission[]

Permission

- - permissionID : String
- + module : String
- + allowedOperation : String
- - createdAt : DateTime

ActivityLog

- - logID : String
- - eventType : String
- - actorID : String
- - resourceRef : String
- - timestamp : DateTime
- + isAnomalous : Boolean

Report

- - reportID : String
- - category : String
- - dateRangeStart : Date
- - dateRangeEnd : Date
- - generatedAt : DateTime
- - generatedBy : String
- + exportFormat : String

DonationRecord

- - recordID : String
- - resourceType : String
- - quantity : Float
- - lifecycleStartDate : Date
- - lifecycleEndDate : Date
- # donationRef : FoodDonation

DatabaseRecord

- - recordID : String
- - recordType : String
- - owningModule : String
- - createdAt : DateTime
- - lastModifiedAt : DateTime
- + isArchived : Boolean

BackupLog

- - backupID : String
- - backupTimestamp : DateTime
- - scope : String
- - outcome : String
- - checksum : String
- - fileLocation : String

AuditLog

- - auditID : String
- - adminID : String
- - operationType : String
- - recordAffected : String
- - timestamp : DateTime

- + isHighRisk : Boolean

Notification

- - notificationID : String
- - eventType : String
- - messageText : String
- - deliveryTimestamp : DateTime
- + deliveryStatus : String
- - channel : String
- # recipientRef : User

TrackingLog

- - logID : String
- - stageName : String
- - transitionTimestamp : DateTime
- - responsibleActor : String
- + isDelayed : Boolean
- # donationRef : FoodDonation

StatusHistory

- - historyID : String
- - statusValue : String
- - enteredAt : DateTime
- - exitedAt : DateTime
- # logRef : TrackingLog

InspectionReport

- - reportID : String
- + outcome : String
- - detailedNotes : String
- - photoEvidenceRef : String
- - inspectedAt : DateTime
- # inspectorRef : FoodInspector
- # donationRef : FoodDonation

FoodSafetyRule

- - ruleID : String
- - category : String
- - maxTemperatureCelsius : Float

- - minShelfLifeDays : Integer
- + isActive : Boolean
- - version : Integer
- - lastUpdatedAt : DateTime

6. State-Dependent Behavior

6.1 Overview

State-dependent behaviour models how an object responds differently depending on its current state. The FoodDonation entity (representing the Docker resource lifecycle) exhibits the most formally defined state machine in this system and is the primary subject of this section. Additional state models are provided for PickupTask and DeliveryTask.

6.2 FoodDonation (Docker Resource) — State Model

State	Description	Entry Trigger	Valid Next States
Submitted	The resource has been submitted by the Donor and is awaiting food safety inspection. It is visible to NGO operators but not yet actionable.	Donor submits via AddDonationFormUI	Inspected
Inspected	An assigned Food Safety Inspector has completed the inspection and recorded an outcome (Safe, Unsafe, or Requires Review). The NGO may now accept or reject the resource.	InspectionController records InspectionReport	Accepted, Rejected, Disposed (if Unsafe)
Accepted	The NGO has reviewed the inspection outcome and formally accepted the resource. The pickup assignment workflow is now initiated.	NGO confirms via DonationDecisionUI	Pickup Assigned
Rejected	The NGO has rejected the resource following inspection. The Donor is notified. The lifecycle terminates unless the Donor resubmits.	NGO rejects via DonationDecisionUI	(terminal or Submitted on resubmit)

Pickup Assigned	A Volunteer has been assigned to collect the resource. The system notifies the Donor and Volunteer.	TaskAssignmentController assigns PickupTask	Picked Up
Picked Up	The Volunteer has physically collected the resource and confirmed pickup in the system. The delivery phase begins.	PickupStatusController updates status	Delivered
Delivered	The Delivery Partner has confirmed successful delivery to the Beneficiary. The donation is now complete.	DeliveryController confirms delivery	Reconciled
Reconciled	The donation record has been matched against the delivery confirmation. The lifecycle is complete and the record is archived.	TrackingController flags completion	(terminal — archived)
Disposed	The resource was flagged as unsafe and has been safely disposed of. The lifecycle terminates at this stage.	UnsafeFoodController confirms disposal	(terminal)

6.3 FoodDonation State Diagram (Textual)



```

+-----+ +-----+
| ACCEPTED | | REJECTED | (terminal / resubmit)
+-----+ +-----+

|
| [ Volunteer assigned ]
|
v
+-----+
| PICKUP ASSIGNED|
+-----+

|
| [ Volunteer confirms collection ]
|
v
+-----+
| PICKED UP |
+-----+

|
| [ Delivery Partner confirms delivery ]
|
v
+-----+
| DELIVERED |
+-----+

|
| [ Tracking Controller completes reconciliation ]
|
v
+-----+
| RECONCILED | (terminal - archived)
+-----+

```

6.4 PickupTask — State Model

State	Description	Trigger
Assigned	A Volunteer has been assigned but has not yet accepted the task.	TaskAssignmentController creates assignment
Accepted	The Volunteer has confirmed acceptance of the task.	PickupTaskController processes acceptance
En Route	The Volunteer is travelling to the donor's pickup location.	PickupStatusController updates status
Arrived	The Volunteer has reached the pickup location.	PickupStatusController updates status
Picked Up	The Volunteer has physically collected the resource.	PickupStatusController updates status

Completed	The resource has been handed off to the Delivery Partner.	PickupStatusController marks complete
Cancelled	The task was cancelled due to an issue or no-show.	IssueController or NGO action

```

Assigned -> Accepted -> En Route -> Arrived -> Picked Up -> Completed
                                   |
                                   [ Issue reported ] -> Cancelled

```

6.5 DeliveryTask — State Model

```

Assigned -> Accepted -> In Transit -> [Delayed] -> Delivered -> Confirmed

```

7. Modeling Inheritance Relationships

7.1 Overview

Inheritance in Object-Oriented Analysis models generalisation-specialisation relationships where a subclass inherits all attributes and operations of its parent class and extends them with specialised behaviour. Two principal inheritance hierarchies are identified in this system: the User hierarchy and the Resource hierarchy.

7.2 User Hierarchy

All actors in the system are specialisations of the abstract User class. The User class defines common identity, authentication, and command-issuance behaviour. Subclasses extend this with role-specific operations.

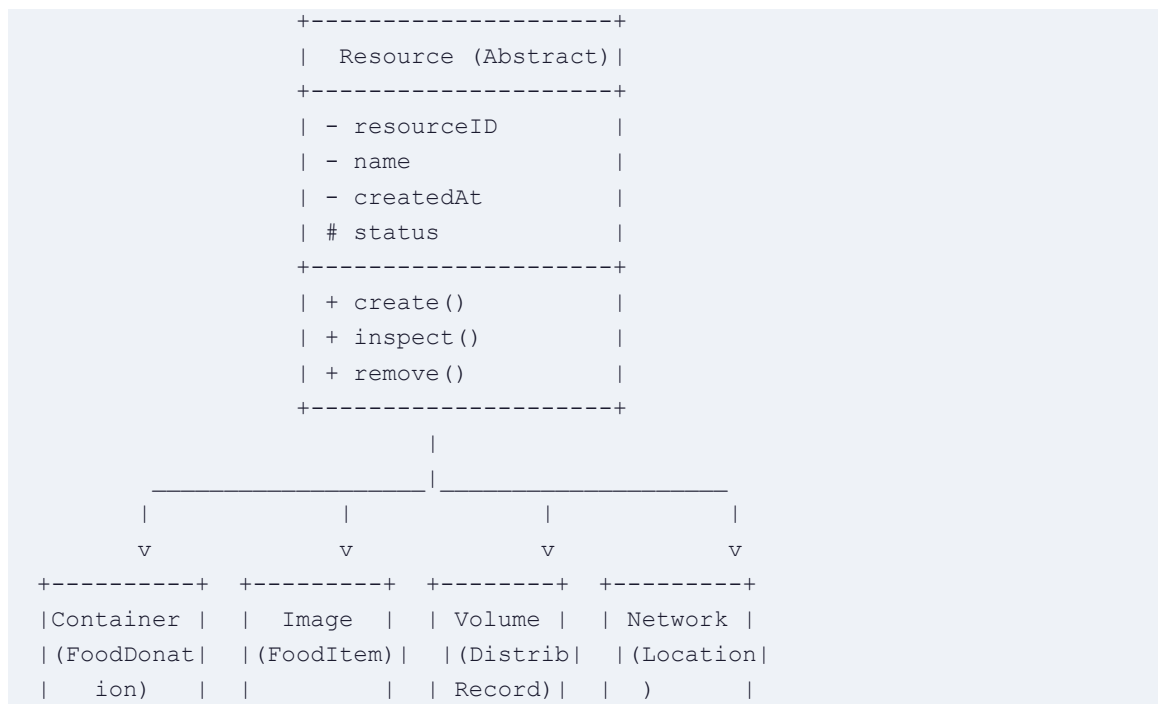
Class	Parent Class	Inherited + Extended Behaviour
User (abstract)	—	Defines: - userID, username, passwordHash, role, isActive. Operations: + authenticate(), + executeCommand(), + setContext(), + viewLogs().
Developer	User	Inherits all User attributes and operations. Extends with: + buildImage(), + writeDockerfile(), + composeUp(), + composeDown(), + pushImage(), + pullImage().
System Administrator	User	Inherits all User attributes and operations. Extends with: + manageDaemon(), + configureEngine(), + setResourceLimits(), + pruneSystem(), + manageUsers(), + inspectAllResources().

7.4 Resource Hierarchy

All Docker-managed runtime resources share common lifecycle operations: creation, inspection, and removal. These are abstracted into a Resource base class from which Container, Image, Volume, and Network are derived.

Class	Parent Class	Inherited + Extended Behaviour
Resource (abstract)	—	Defines: - resourceID, name, createdAt, status. Operations: + create(), + inspect(), + remove().
Container (FoodDonation)	Resource	Inherits Resource. Extends with: status, imageName, ports, environment. Operations: + start(), + stop(), + pause(), + exec(), + logs().
Image (FoodItem)	Resource	Inherits Resource. Extends with: tag, size, layers, architecture. Operations: + build(), + pull(), + push(), + tag(), + prune().
Volume (DistributionRecord)	Resource	Inherits Resource. Extends with: driver, mountPath, scope. Operations: + mount(), + unmount(), + prune().
Network (Location)	Resource	Inherits Resource. Extends with: driver, subnet, gateway, attachedContainers. Operations: + connect(), + disconnect(), + prune().

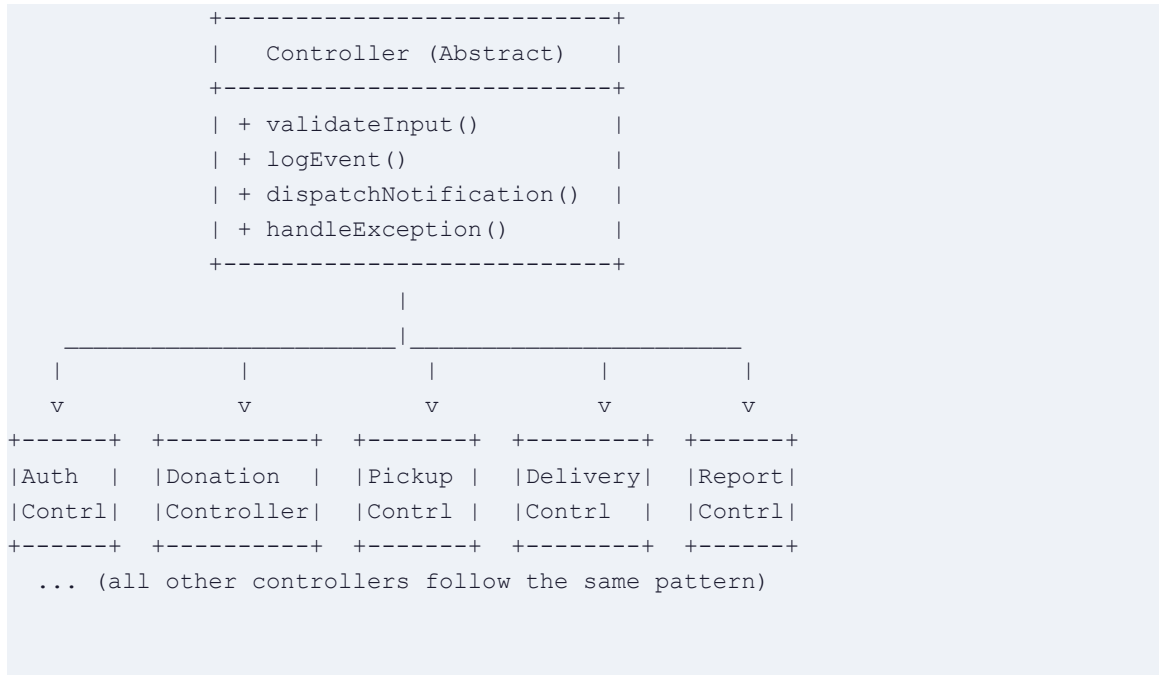
7.5 Resource Inheritance Diagram (Textual)



+-----+	+-----+	+-----+	+-----+
+start()	+build()	+mount()	+connect()
+stop()	+pull()	+unmount	+disconn()
+pause()	+push()	+prune()	+prune()
+exec()	+tag()	+-----+	+-----+
+logs()	+-----+		
+-----+			

7.6 Controller Hierarchy

All Control objects inherit from a base Controller class that provides common cross-cutting operations such as transaction management, event logging, and notification dispatching.



Food Redistribution System - Detailed System-Wide Class Diagram

